

Actividad API REST (Ionic + Angular Standalone) — entrega en progreso

Jorge González Gómez · 2º DAM · Desarrollo de Interfaces · 06-10-2026 (versión 1)

GitHub: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/03-API-REST-Productos · Vercel: <https://desarrollo-interfaces-dam-2-qeef.vercel.app>

Entrega parcial. Ahora mismo estoy repasando la base de Angular (data binding, signals) y los Observables para entender bien lo que hago antes de seguir. Iré completando las peticiones pendientes y actualizando GitHub y este documento durante la semana.

Hecho

- Base de la actividad: modelo, servicio con `HttpClient`, rutas, Inicio y tabla de productos de dummyjson con estados de carga/error/reintento.
- **1.** Ancho × alto en la tabla.
- **2.** Stock valorado (unidades × precio – descuento).
- **3.** Botón "Volver a inicio".
- **5, 7 y 8** explicados abajo.

Pendiente

- **4.** Página About con reseña y enlace a GitHub.
- **6.** Paginación.
- **9.** Botón modo oscuro.
- **10.** Vista con cards/dashboard en vez de tabla.

Lo más interesante: con `HttpClient` no hay `async/await`

`HttpClient` devuelve un **Observable**, no una Promise: la petición no se envía hasta que alguien se suscribe. Por eso el servicio solo "prepara" la petición y la página se suscribe con `next` / `error`.

```
// product.service.ts
private http = inject(HttpClient); // inyección de dependencias
getProducts(): Observable<ProductsResponse> {
  return this.http.get<ProductsResponse>('https://dummyjson.com/products');
}

// productos.page.ts
this.productService.getProducts().subscribe({ // aquí se envía la petición
  next: (response) => { this.products.set(response.products); this.loading.set(false); },
  error: () => { this.error.set('No se han podido cargar los productos.');
```

Petición 2: stock valorado

```
stockValorado(p: Product): number {
  const bruto = p.stock * p.price; // unidades * precio
  return bruto - bruto * (p.discountPercentage / 100); // menos el % de descuento
}

<!-- en la tabla -->
<td>{{ product.dimensions.width }} × {{ product.dimensions.height }}</td>
<td>{{ stockValorado(product) | currency: 'EUR' }}</td>
```

Ejemplo: 99 uds × 9,99 € = 989,01 €, con un 10 % de descuento → 890,11 €.

Petición 7: ¿por qué existe app.config.ts?

En Angular standalone ya no hay `AppModule`. La configuración global (router, Ionic y `provideHttpClient()`, que hace falta para poder inyectar `HttpClient`) va en un objeto `ApplicationConfig` en `app.config.ts`, y `main.ts` arranca con `bootstrapApplication(AppComponent, appConfig)`. La plantilla de Ionic no lo trae, así que lo creé yo.

Petición 8: de la rama desarrollo a main

Trabajo en `desarrollo`; cuando algo está probado lo paso a `main`, que es la rama que publica Vercel:

```
git checkout desarrollo → git add . → git commit -m "... " → git push origin desarrollo
git checkout main → git merge desarrollo → git push origin main → git checkout desarrollo
```

Petición 5: troubleshooting

- **No compila '@ionic/angular/standalone'**. Mi proyecto usa Ionic 9 y esa ruta ya no existe. Solución: importar de '@ionic/angular'.
- **Se queda en "Cargando productos..." para siempre**. Angular 22 no usa zone.js y no detecta cambios en variables normales dentro del `subscribe`. Solución: guardar el estado en **signals** (`signal()` + `.set()`).
- **404 al recargar /productos en Vercel**. Es una SPA y solo existe `index.html`. Solución: `vercel.json` con un `rewrite` de todas las rutas a `index.html`.